AWS Translator Project Documentation

1. Project Overview

The AWS Translator Web Application demonstrates how multiple AWS services can be integrated to create both a chatbot-based translator and a direct text translation web app.
Initially, the system was implemented using Amazon Lex (chatbot) and AWS Lambda, and later simplified into a frontend-only architecture that communicates directly with Amazon Translate and Amazon Comprehend using Amazon Cognito for secure access.
Both implementations showcase real-world use of AWS AI and serverless services for natural language processing and web deployment.

2. Objectives

- Build a multi-language translator capable of detecting the input language automatically.
- Demonstrate how AWS services can interact through both chatbot and web UI models.
- Deploy the system as a serverless static web application using AWS free-tier components.

3. AWS Services Used

AWS Service	Purpose
Amazon Translate	Performs text translation between languages.
Amazon Comprehend	Auto-detects the input (source) language when “Auto-detect” is selected.
Amazon Cognito	Provides temporary credentials to securely access AWS services from the browser.
Amazon S3	Hosts the static web application (HTML, CSS, and JavaScript).
AWS IAM	Manages permissions for unauthenticated users accessing Translate/Comprehend.
Amazon Lex (Phase 1)	Used to create a chatbot interface for translation via text interaction.
AWS Lambda (Phase 1)	Acts as the backend logic for Lex—invoking Amazon Translate through Python code.

Amazon CloudWatch	Collected logs and metrics for the Lambda function. Helped debug translation responses and monitor function execution during Phase 1.
AWS CloudShell	Browser-based command-line tool used to run AWS CLI commands, check IAM configurations, create resources, and test policies during setup and debugging.

4. System Architecture

Phase 1: Chatbot-Based Translator

In the initial stage, the project implemented a chatbot translator using Amazon Lex and Lambda.

Workflow:

1. User typed phrases like *"Translate to Hindi"* or *"Translate hello to Spanish"* into the Lex test console or chat UI.

2. Amazon Lex recognized the user's intent and passed the input text and language choices to a Lambda function.

3. The Lambda function (Python) uses Amazon Translate to translate the text, optionally auto-detecting the source language.

4. Lex returned the translated message back to the user.

Architecture Flow:

User → Lex Bot → Lambda → Amazon Translate → Lex → User

Purpose of this phase:

- Demonstrate natural language understanding via Lex Intents and Slots.

- Show how Lambda can integrate AWS AI services into a conversational workflow.

- Serve as a demo of chatbot-based translation for interactive use cases.

Phase 2: Direct Web Translator (Final Version)

After successfully building the chatbot version, the architecture was optimized for a frontend-only web translator using Cognito + Amazon Translate + Comprehend.

Workflow:

1. The user opens the translator web app hosted on Amazon S3.
2. The web app requests temporary AWS credentials from Amazon Cognito Identity Pool (Unauthenticated role).
3. The app detects the language using Amazon Comprehend if “Auto-detect” is selected.
4. The app calls Amazon Translate directly from the browser using AWS SDK.
5. The translated text is displayed instantly in the UI.

Architecture Flow:

Browser (app.js) → Cognito → Amazon Comprehend → Amazon Translate → Browser (Result)

Advantages of this phase:

- No Lambda or Lex dependency (fewer AWS components).
- Instant translation, reduced latency.
- Lower cost and simpler maintenance.
- Fully static deployment — no backend required.

5. Implementation Details

Phase 1 – Chatbot Translator (Lex + Lambda + Translate)

Starting Point: IAM Role Configuration

Before integrating any service, an IAM role was created and attached to the Lambda function to define what AWS resources the function could access.
The Lambda execution role was configured with permissions for Amazon Translate and Amazon Comprehend, allowing the function to detect and translate languages on behalf of the user.

Step-by-Step Implementation

1. Created an Amazon Lex V2 Bot
    - Defined an intent named TranslateIntent.
    - Added utterances like “Translate to Hindi” or “Translate hello to French.”
    - Set up optional slots for source and target languages.
    - Enabled the Fulfillment code hook and linked it to the Lambda function.
2. Created an AWS Lambda Function (Python)
    - The function received the input text from Lex, called Amazon Translate, and returned the translated output.
    - If the source language was not specified, it used Comprehend for automatic language detection.
3. Connected Lex and Lambda
    - Linked the Lambda under the Fulfillment code hook of the intent.
    - Built and deployed the bot alias (for example, TestBotAlias).
4. Testing and Verification
    - Used the Lex Test console to enter text (e.g., “Hello, how are you?”).
    - Verified that the translated text was returned correctly from Lambda.
    - Checked the CloudWatch logs to ensure Lex triggered Lambda successfully.

Result:
The chatbot could translate messages directly inside the Lex console using Lambda + Translate, simulating an intelligent multilingual assistant.
However, it required multiple services (Lex, Lambda, Translate) and had higher latency due to the multi-step call chain.

Phase 2 – Direct Web Translator (Cognito + Translate + Comprehend)

Starting Point: IAM Role for Cognito Unauthenticated Identities

In the second phase, the architecture was simplified to remove Lex and Lambda. Instead, the IAM role assigned to Cognito unauthenticated users was configured to allow frontend access to Amazon Translate and Comprehend directly from the browser.

Amazon Cognito Identity Pool Setup

- Created an Identity Pool with unauthenticated access enabled.
- Copied the Identity Pool ID into app.js.
- Verified the role mapping so that unauthenticated users assume the Cognito_lexWebChatUnauthRole.

Frontend Development (index.html & app.js)

- Designed a responsive translator UI using HTML and CSS.
- Implemented AWS SDK for JavaScript (v2) for Translate and Comprehend.
- Added interactive elements:
    - Text area for input
    - Dropdowns for source and target languages
    - Buttons: Translate, Swap, Copy, and Clear

S3 Hosting

- Static website hosting enabled.
- Uploaded index.html and app.js.
- Website accessible via S3 public endpoint URL.

6. Challenges and Solutions

- IAM Permission Errors
  Faced AccessDeniedException when connecting the frontend to Amazon Translate.
  Solved by adding translate:TranslateText and comprehend:DetectDominantLanguage permissions to the Cognito unauthenticated role.

- Lambda Access Denied (Chatbot Phase)
  Lambda could not call Comprehend due to missing permissions.
  Fixed by editing the Lambda execution role to include Translate and Comprehend actions.

- Parameter Validation Error
  The Lambda failed because SourceLanguageCode was missing.
  Resolved by adding SourceLanguageCode="auto" to handle automatic detection properly.

- Transition to Direct Web Translator
  Lex and Lambda made translation slower and complex.
  Simplified the architecture by removing them and calling Translate directly via Cognito and AWS SDK.

- SDK Connection Issues
  The app failed to connect due to incorrect region and cached credentials.
  Corrected the region configuration (us-east-1) and cleared cached AWS credentials in the SDK.

- Auto-Detect Not Working
  Translate occasionally failed to detect the source language automatically.
  Integrated Amazon Comprehend to reliably identify and pass the detected language to Translate.

- UI Overlap and Missing Buttons
  Buttons and text areas were too close together, making the interface cluttered.
  Adjusted CSS layout, added Clear and Swap buttons, and improved spacing for better usability.

- Cached Old Files on S3
  After updates, older versions of files still appeared on the hosted site.
  Re-uploaded updated files to S3 and performed a hard refresh (Ctrl + F5) to clear cache.

- Confusion Between IAM and Cognito Roles
  Initially unclear how IAM roles and Cognito credentials interacted.
  Learned that IAM defines access policies while Cognito provides temporary credentials for secure browser access.

- Debugging Errors in Lambda
  It was difficult to locate issues during chatbot testing.
  Used Amazon CloudWatch logs to trace Lambda execution and AWS CloudShell to test roles and permissions.

7. Final Results

The translator app successfully:

- Detects the input language automatically.
- Translates text into 12 languages.
- Allows copying, clearing, and swapping of languages.
- Works entirely in the browser (serverless).

8. Key Learnings

- How to integrate multiple AWS AI services in one workflow.
- Difference between chatbot workflows (Lex + Lambda) and direct browser integration.
- Using Cognito for secure frontend AWS access.
- Building and deploying static web apps on S3.
- Implementing auto-detect logic with Amazon Comprehend.

9. Future Enhancements

- Add image translation using Amazon Textract + Translate.

10. Conclusion

This project demonstrates two approaches to implementing translation systems using AWS:

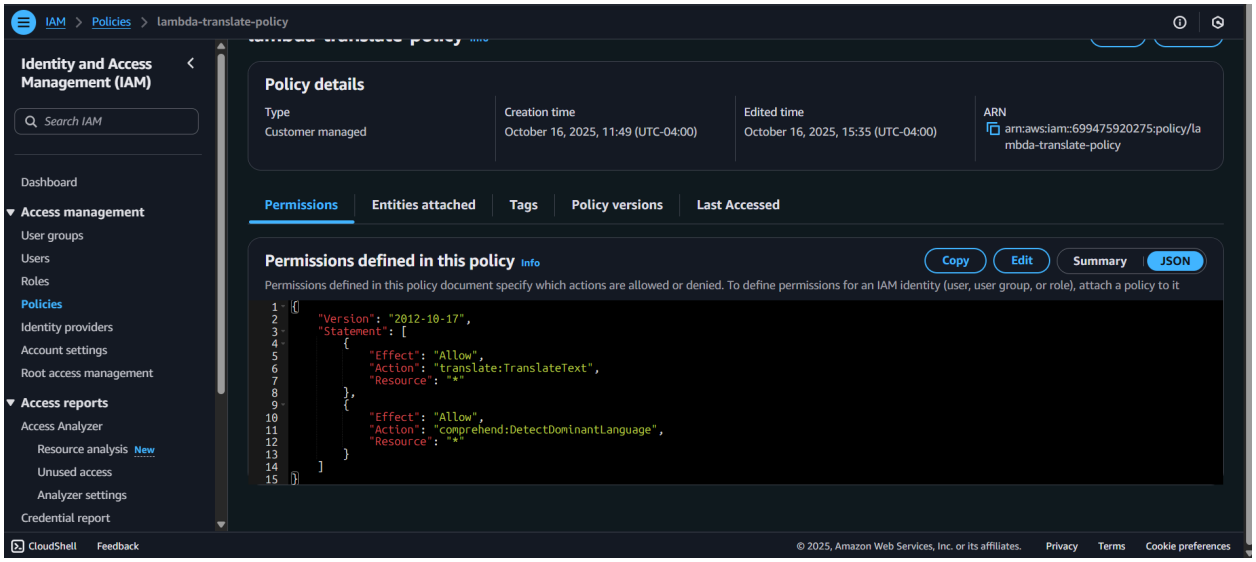
1. Chatbot Version (Lex + Lambda) for interactive, conversational translation.

2. Frontend Web App Version (Cognito + Translate + Comprehend) for fast, serverless translation.

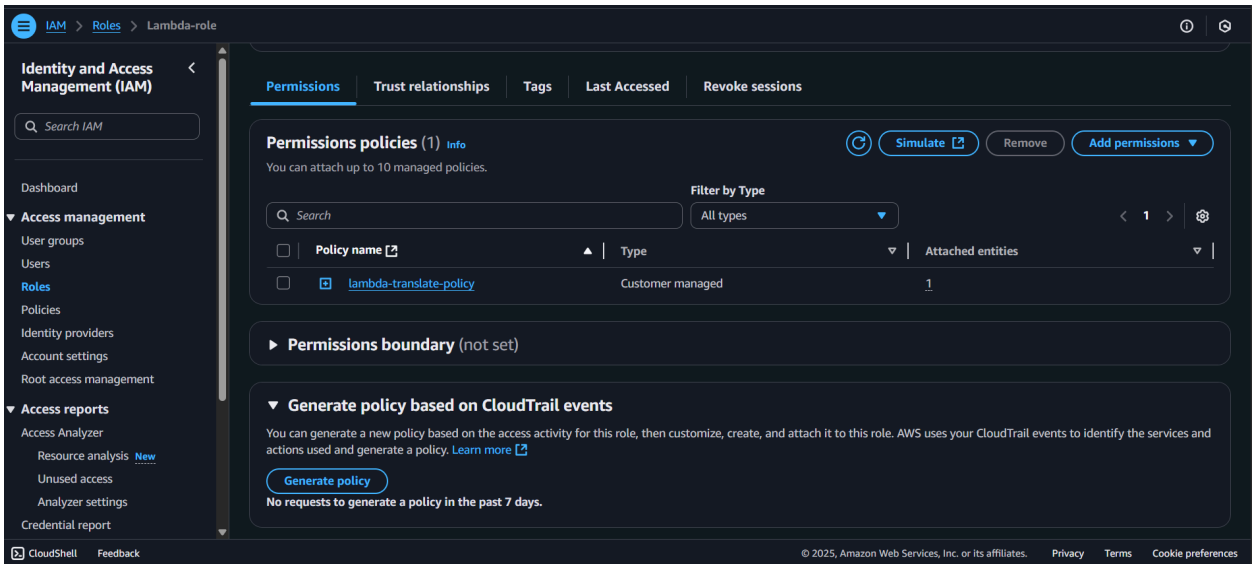
The final system is lightweight, cost-efficient, and user-friendly showcasing the power of AWS AI and serverless architecture for real-world multilingual applications.

Snapshots of Project:

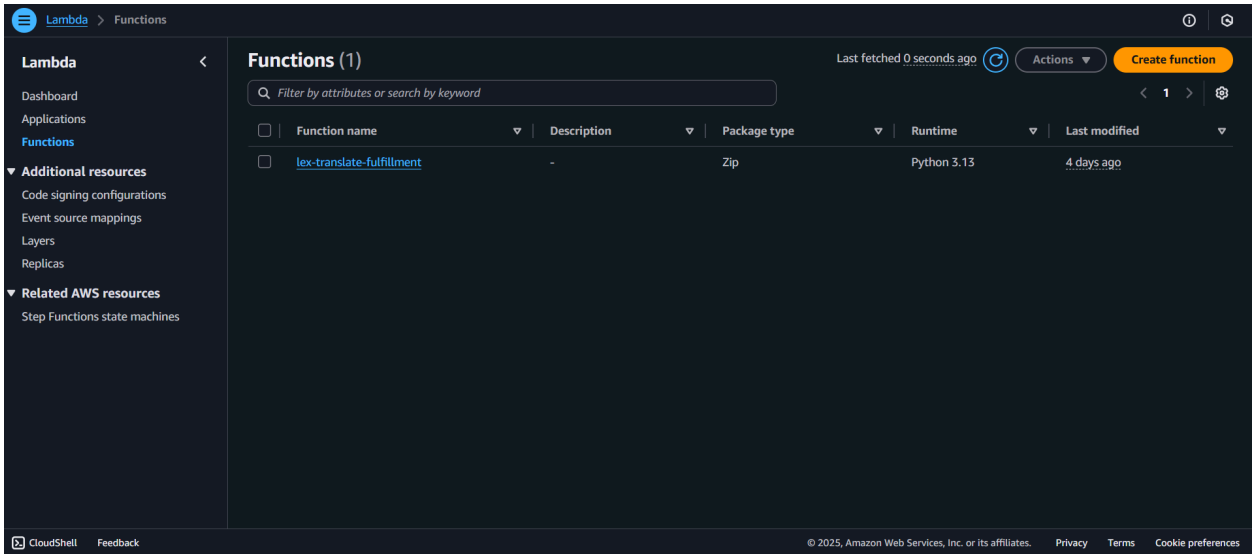
Lambda-translate-policy

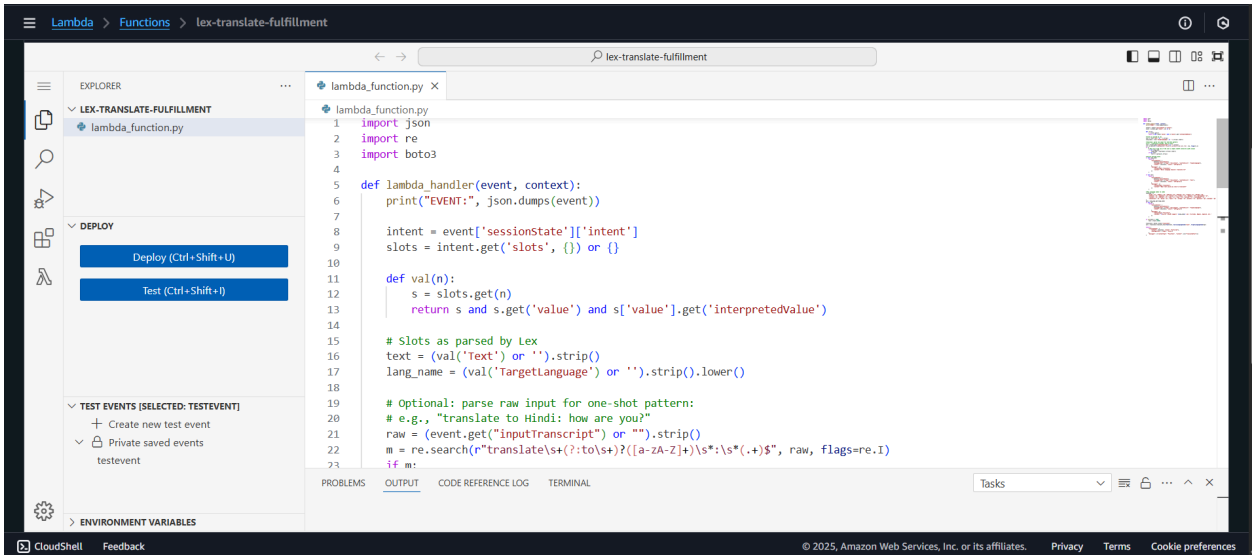


Lambda-role

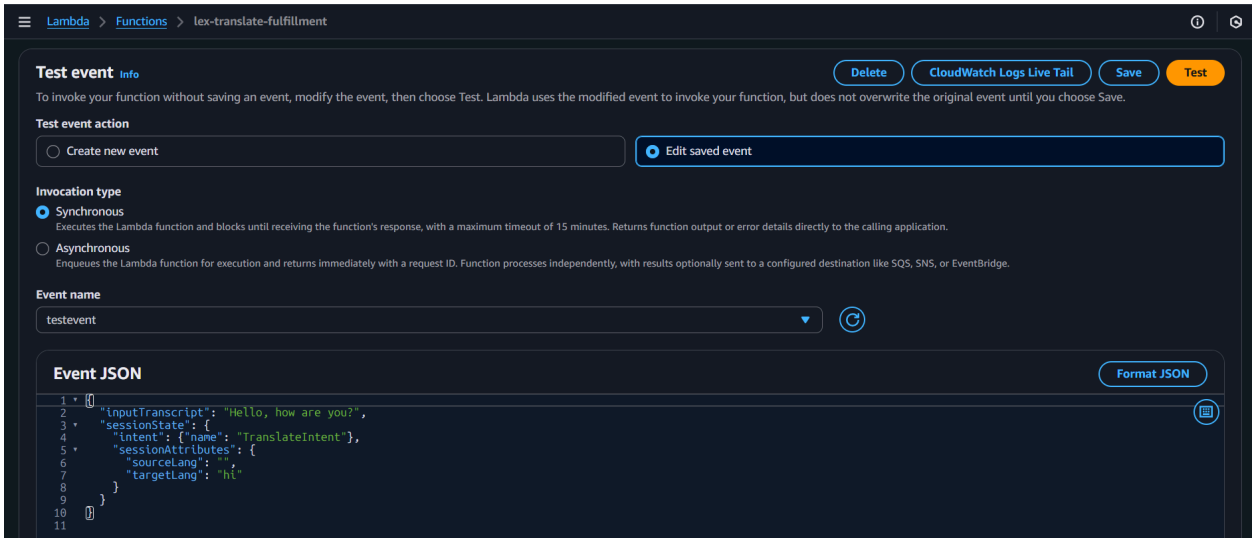


Lambda function:

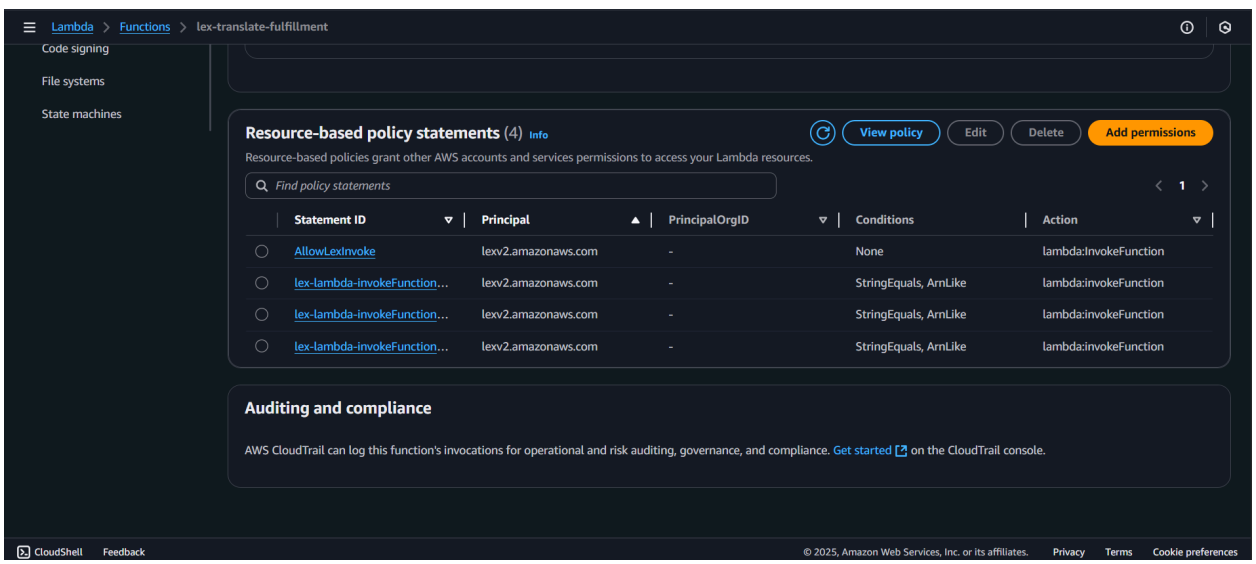




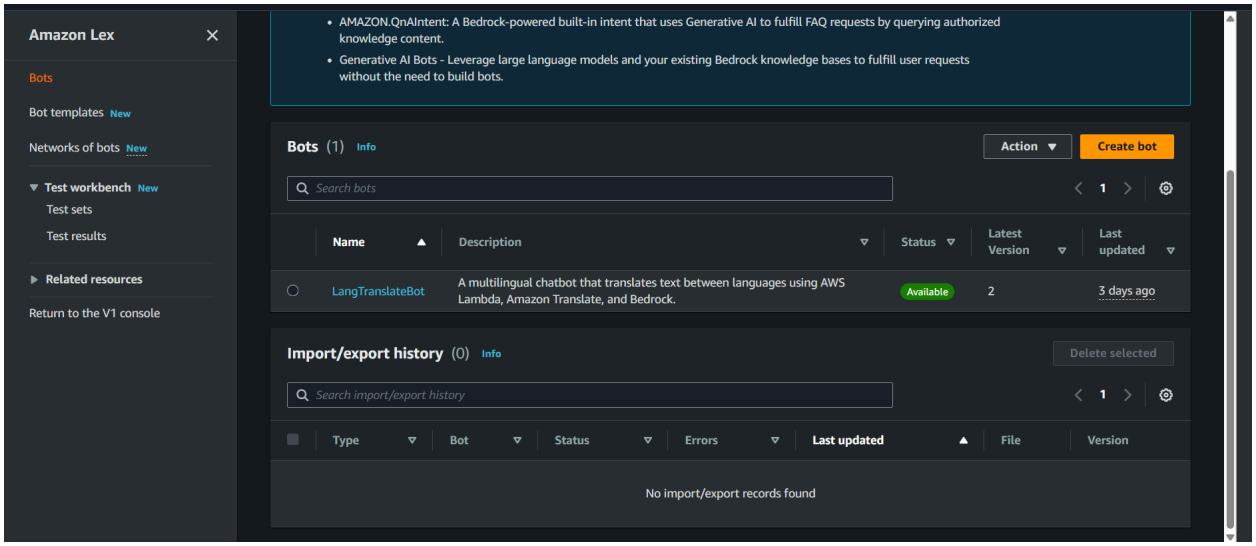
Test event for Lambda function



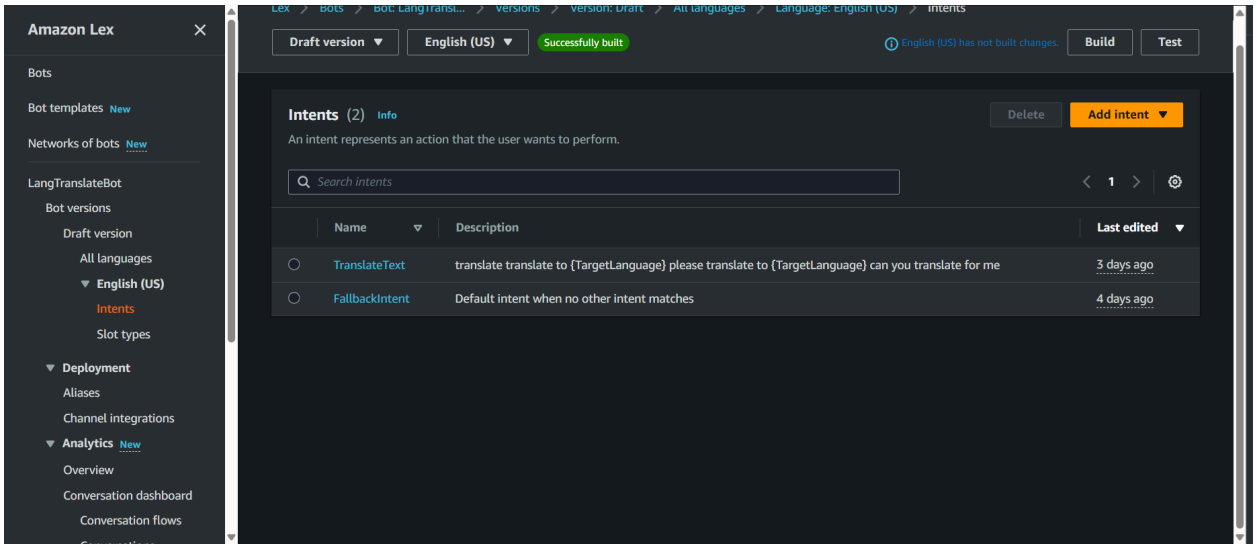
Permissions for Lambda function

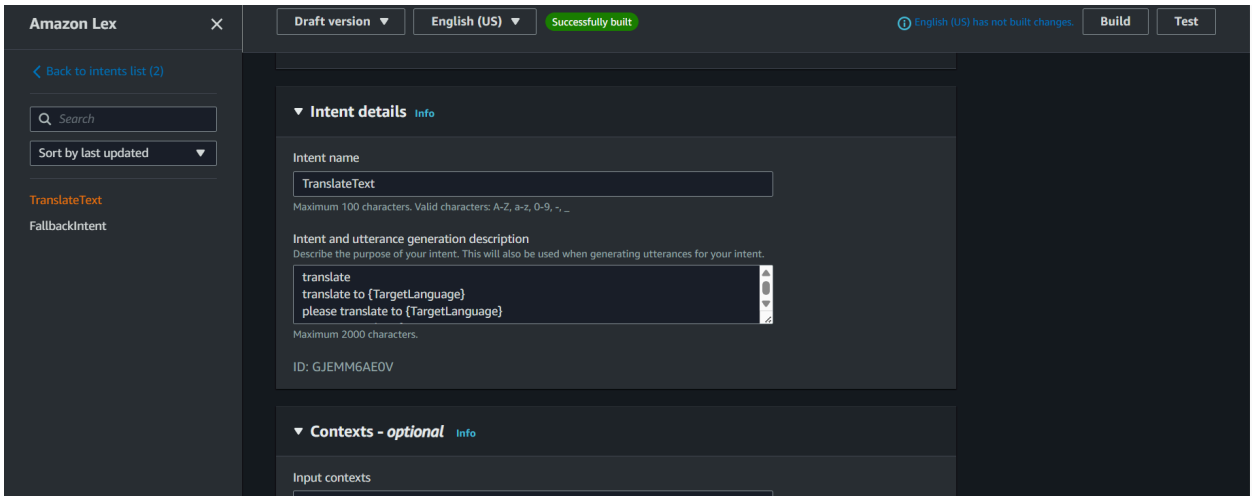


Amazon Bot created

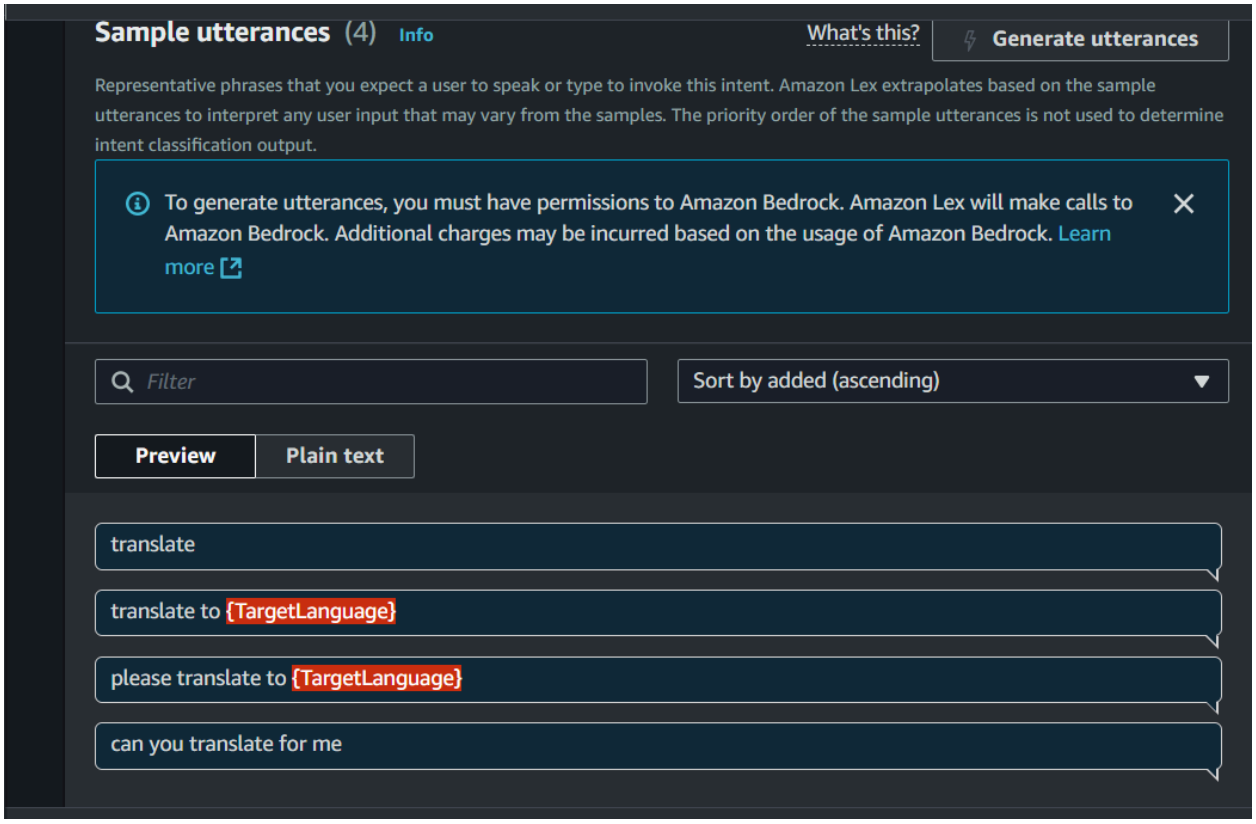


Intents created

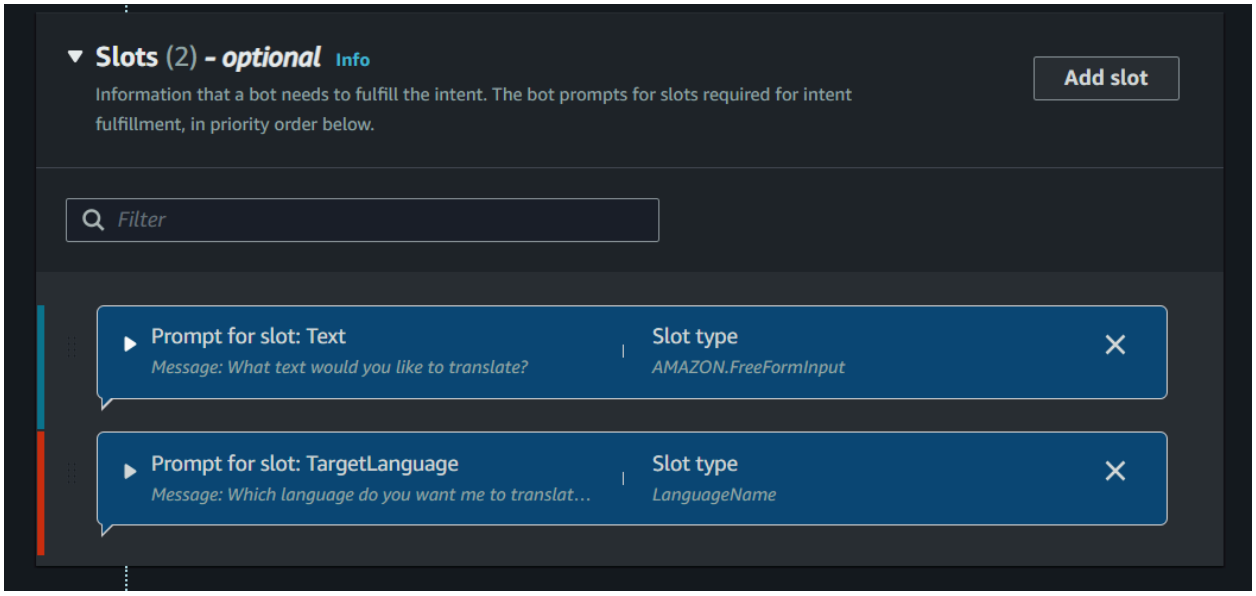




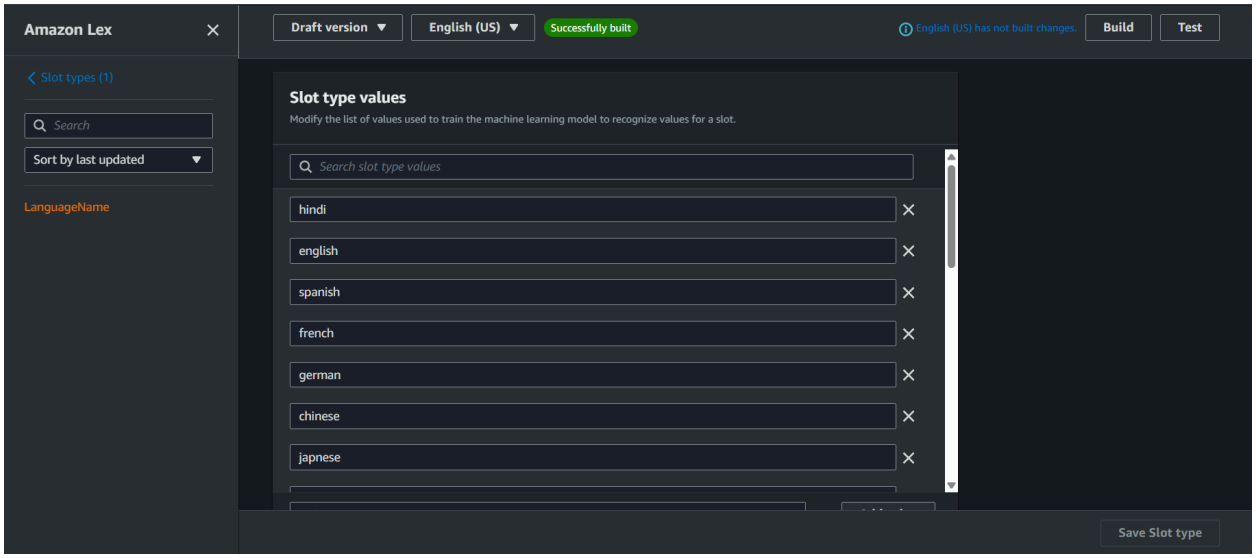
Sample utterances in intent



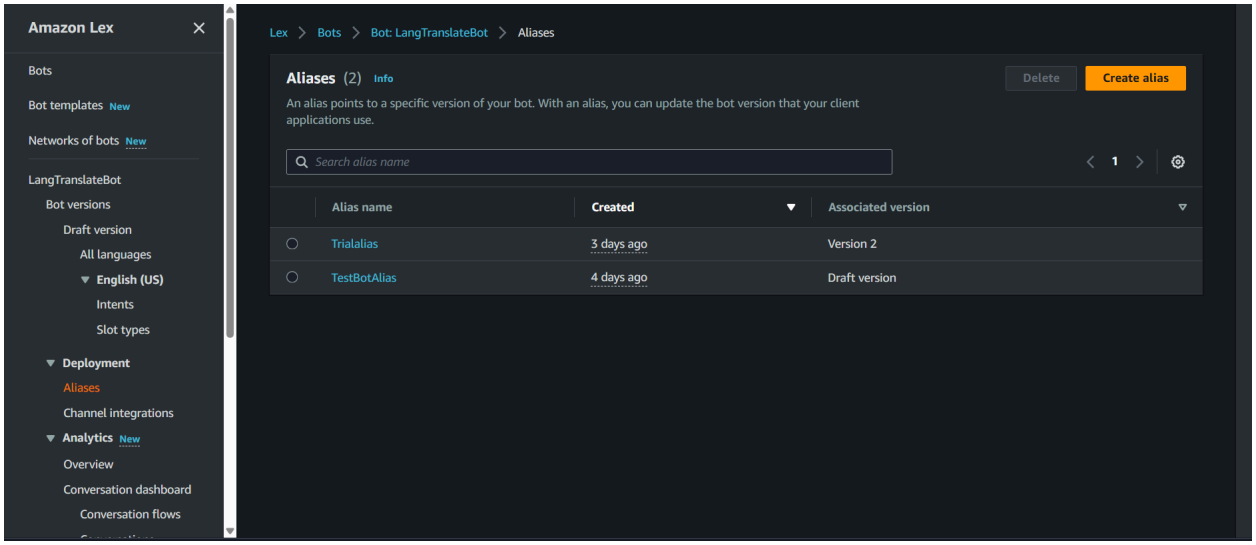
Slots in intent



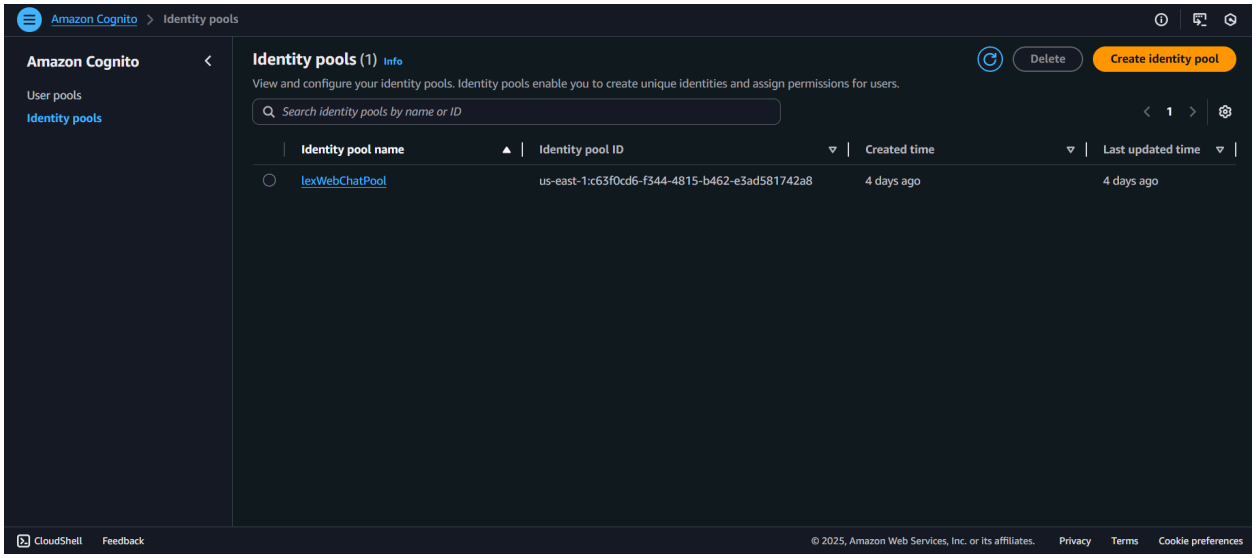
LanguageName Slot creation



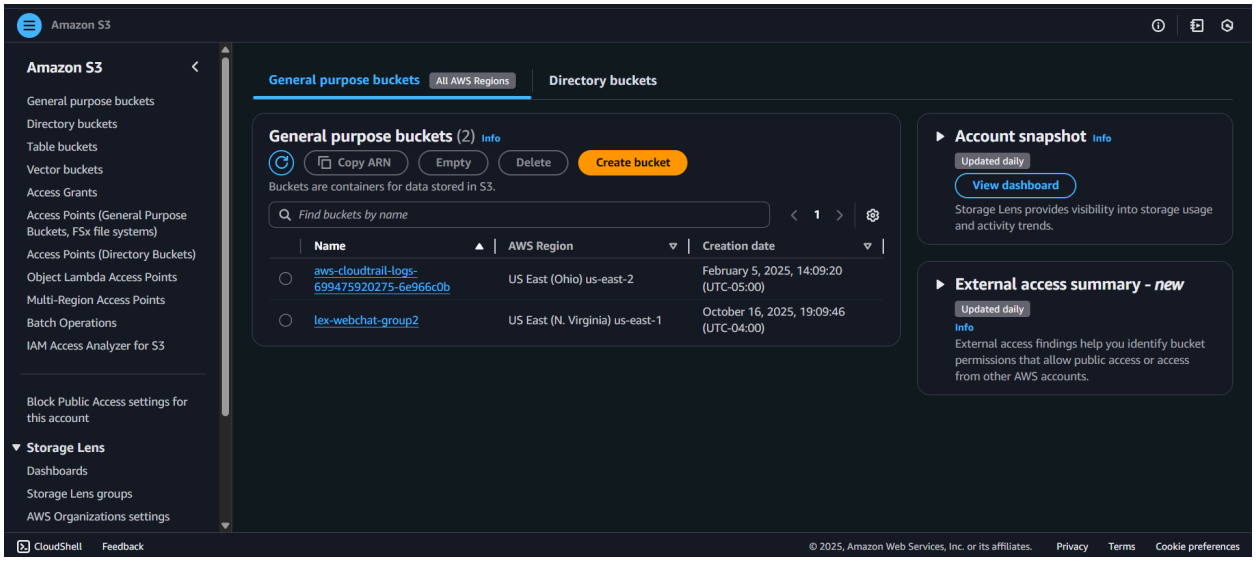
Created Aliases



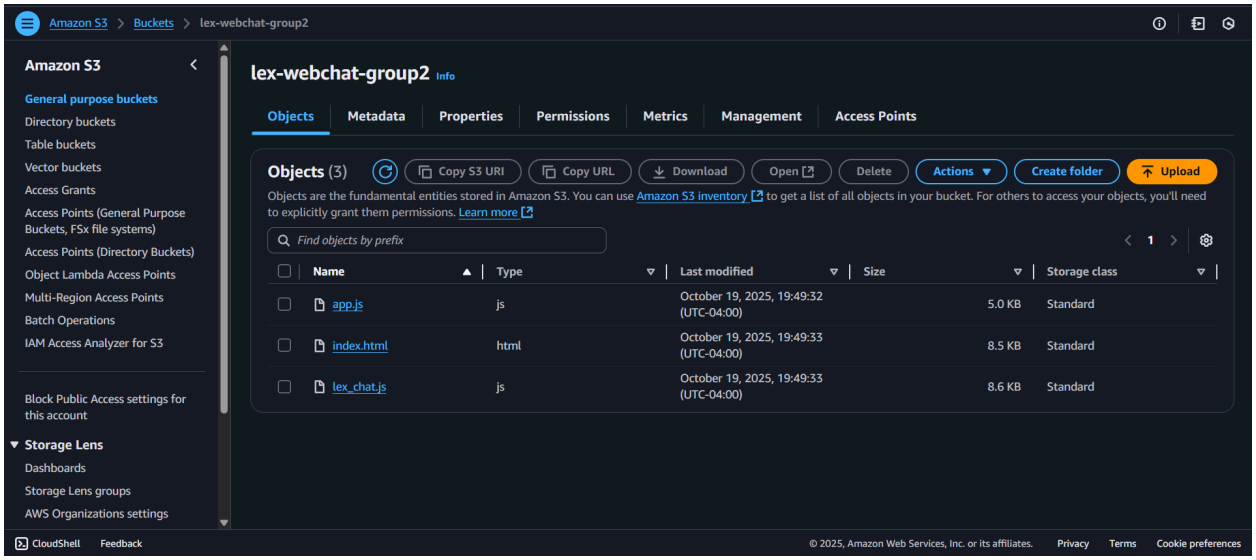
Created identity pool in cognito



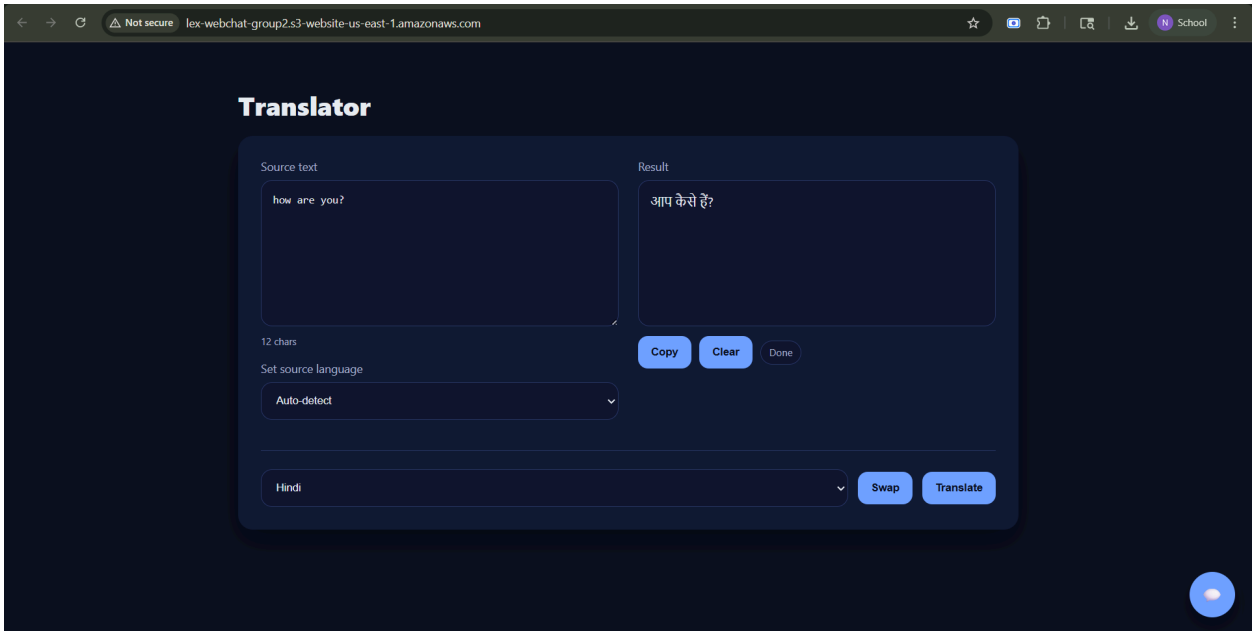
Created Bucket S3



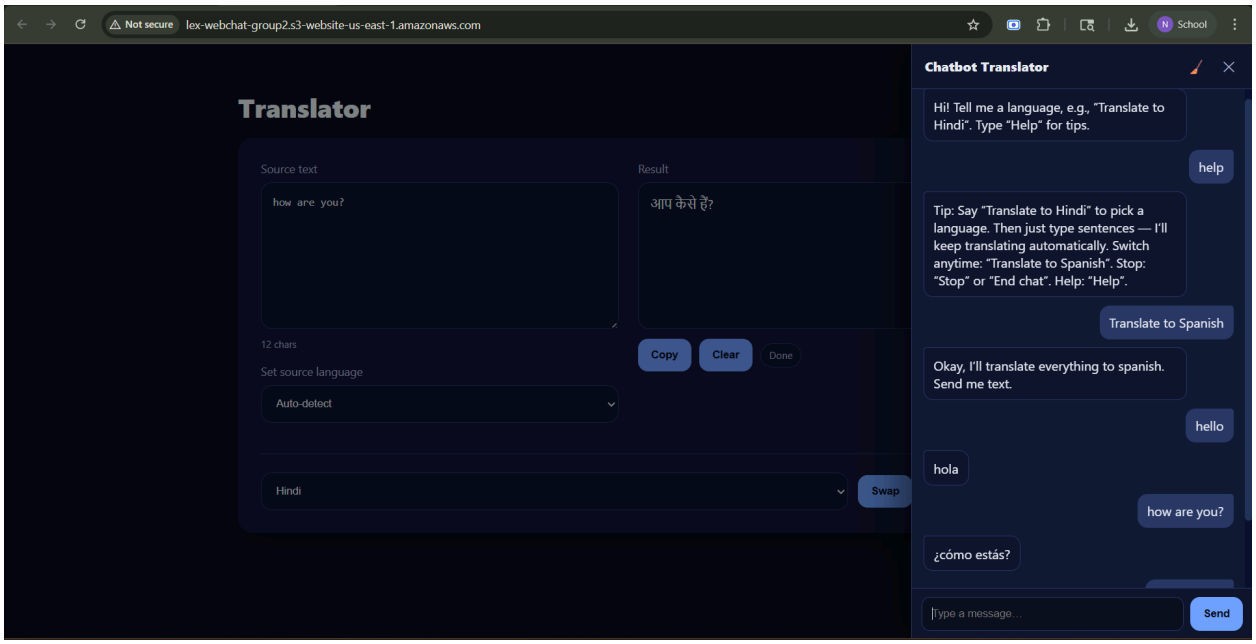
Added files on it



UI: Default



UI: Chatbot


Not secure
lex-webchat-group2.s3-website-us-east-1.amazonaws.com
School
Translator
Source text
how are you?
12 chars
Set source language
Auto-detect
Result
आप केसे हैं?
Copy
Clear
Done
Hindi
Swap
Chatbot Translator
Hi! Tell me a language, e.g., "Translate to Hindi". Type "Help" for tips.
help
Tip: Say "Translate to Hindi" to pick a language. Then just type sentences — I'll keep translating automatically. Switch anytime: "Translate to Spanish". Stop: "Stop" or "End chat". Help: "Help".
Translate to Spanish
Okay, I'll translate everything to spanish. Send me text.
hello
hola
how are you?
¿cómo estás?
Type a message...
Send